2026학년도 2학기 언어데이터과학

제17강 코퍼스 구축과 웹 크롤링

박수민

서울대학교 인문대학 언어학과

2026년 11월 11일 수요일

지난 시간에 배운 것

1. `load_dataset`으로 저장소, 설정, 분할, 커밋을 정해 불러왔다.
2. 데이터를 만든 과정이 수에 남는다(모순 가설의 부정 표현 29.4% 대 14.6%).
3. 일곱 가지 물음의 점검표로 남의 데이터를 살폈다.

오늘의 목표

1. 연구 물음에 맞는 자료원을 찾고, 얻는 절차와 쓰는 조건을 비교할 수 있다.
2. 모으기 전에 `robots.txt`, 약관, 라이선스, 개인정보를 확인할 수 있다.
3. `requests`와 `BeautifulSoup`으로 웹 페이지를 모아 표로 만들 수 있다.
4. 내가 만든 코퍼스의 카드를 써서, 남이 다시 할 수 있게 남길 수 있다.

사전 준비

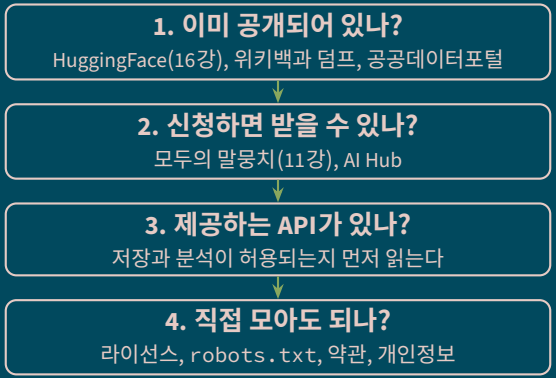
터미널 입력 (Codespaces)

```
pip install requests beautifulsoup4
mkdir -p data/wikisource/raw corpus/wikisource
```

노트북 17-corpus.ipynb의 첫 셀 — 마지막 줄은 16강의 그 줄이다

```
import os, re, time
from glob import glob
from urllib import robotparser
import pandas as pd
import requests
from bs4 import BeautifulSoup
from tqdm import tqdm
pd.set_option('mode.string_storage', 'python')
```

어디서 구할까



위에서 아래로 내려간다

아래로 갈수록 표본 설계, 권리 확인, 품질 점검이 내 몫이 된다. 크롤링은 **마지막 수단**이다.

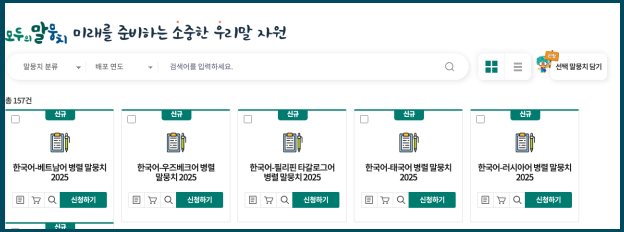
1 **자료원 둘러보기**
신청해서 받는 말뭉치
내려받는 공개 자료
네이버 블로그

2 직접 모으기

3 남이 다시 할 수 있게

4 마무리

모두의 말뭉치



https://kli.korean.go.kr/corpus (2026-11-03 캡처)

[숙제07]에서 가입하고 11강에서 쓴 곳이다. 이제 **157건**이 있다.

말뭉치마다 설명 자료(PDF)와 미리보기가 공개되어 있다. 신청하기 전에 읽을 수 있다.

절차는 11강과 같다

말뭉치를 고르고, 사용 목적을 적어 신청하고, 승인을 받으면 zip을 내려받는다. 승인에는 시간이 걸리니 기말프로젝트에 쓰려면 일찍 신청한다.

온라인 게시 자료 말뭉치

	2025 (버전 1.0)	**2022** (버전 1.0)
생산 시기	2023–2024년	—
게시판	—	8,282건 (14개 매체)
블로그	**네이버 블로그 73,176건**	—
누리 소통망	인스타그램 42,937건	인스타그램, 페이스북 296,892건
합계	116,113건	305,174건
파일	zip 1개, 1.95GB	763개, 123MB

이름이 비슷한 11강의 말뭉치와 다르다

11강의 **온라인 대화** 말뭉치는 두 명 이상이 주고받은 메시지다. 온라인 **게시** 자료는 한 사람이 불특정 독자에게 쓴 글이다. 매체도 말의 쓰임도 다르다.

출처: 국립국어원, 온라인 게시 자료 말뭉치 2025·2022 설명 자료. 영어 이름은 *Online Posting Materials Corpus*.

한 건은 어떻게 생겼나

설명 자료의 예시 — 11강처럼 JSON이다 (줄였다, 아이디는 가렸다)

```
"category": "웹 > 블로그",
"sampling": "게시자 모집 후 무작위 추출",
"title": "&company& 신메뉴 &brand& 톡톡 튀는 상큼함!",
"author": "writer001",
"date": "20230102",
"crawl_date": "20250710 19:36:14",
"url": "https://blog.naver.com/{blogId}/{logNo}",
"form": "2023년이 된지 벌써 2일째에요"
```

sampling 어떻게 뽑았는지가 파일 안에 있다. 글쓴이를 모집했다

writer001 글쓴이의 이름은 가명으로 바꾸었다

&company& 회사와 상표 이름도 자리표시자로 바꾸었다

crawl_date 언제 모았는지 초까지 남겼다. 오늘 우리도 따라 한다

AI Hub



https://www.aihub.or.kr (2026-11-03 캡처)

한국지능정보사회진흥원이 운영하는 AI 학습용 데이터 저장소다. 2026-11-03 현재 목록에 **867개**가 있고, 이미지, 영상, 음성, 텍스트가 섞여 있다.

키	데이터셋	내용
114	한국어 SNS	대화 200만 건, 성별·나이·지역 포함, JSON
625	온라인 구어체 말뭉치	게시판, 댓글 등 10억 어절 이상, 형태소 등 주석

AI Hub에서 받는 방법

터미널 입력 — 목록은 키 없이 볼 수 있다

```
curl -o aihubshell https://api.aihub.or.kr/api/aihubshell.do
chmod +x aihubshell; ./aihubshell -mode l | grep 'SNS'
```

실행 결과

```
114, 한국어 SNS
71343, SNS 데이터 고도화
71694, 한국어 SNS 멀티턴 대화 데이터
```

내려받기 — 승인과 키가 먼저다

```
./aihubshell -mode d -datasetkey 114 -aihubapikey '내 키'
```

페이지에서 [다운로드]로 승인을 받고, 메일로 받은 키를 넣는다. **키는 저장소에 올리지 않는다.**

AI Hub의 조건을 읽는다

이용정책에 적힌 것	내게 뜻하는 것
“인공지능 학습모델의 학습용으로만”	언어 연구도 되는지 적혀 있지 않다. 묻는다
승인 없이 제공, 양도, 대여, 판매 금지	데이터를 남에게 주거나 저장소에 올리지 않는다
“사업결과임을 밝혀야”	보고서에 출처를 적는다
국외 이용은 별도 합의	해외로 가져가려면 먼저 묻는다
“내국인만 데이터 신청이 가능합니다”	데이터셋 페이지의 문구. 페이지마다 확인한다

기말프로젝트에 쓴다면

용도와 신청 자격을 먼저 확인한다. 조건이 맞지 않으면 모두의 말뭉치나 다른 자료원으로 간다. 신청 자격이 걸리는 수강생은 교수자에게 미리 알린다.

출처: AI Hub 이용정책(https://www.aihub.or.kr/intrcn/guid/usagepolicy.do), 2026-11-03 확인.

데이터셋마다 조건이 더 있을 수 있다.

공개 자료를 내려받는 곳

자료원	받는 것	조건
위키백과 덤프	전체 문서 XML, 1.4GB(`.bz2`)	CC BY-SA
공공데이터포털	기관이 올린 CSV, API	공공누리 유형을 데이터마다 본다
정책브리핑	정부 부처의 보도자료	공공누리 유형을 글마다 본다
위키문헌	저작권이 끝난 옛글	작품마다 틀이 있다

긁기 전에 덤프를 찾는다

위키미디어의 로봇 정책도 덤프를 먼저 생각하라고 권한다. XML은 14강에서 읽었다. 공공데이터포털에는 모두의 말뭉치의 통계 목록도 있다.

16강의 KLUE도 이렇게 모았다

KLUE의 `policy`는 정부 기관의 글로, 공공누리 제1유형이라 골랐다고 논문에 적혀 있다.

네이버 블로그를 쓰고 싶다면

탐나는 자료다

많은 사람이 일상의 말로 쓰고, 날짜가 붙어 있고, 주제도 다양하다. 그런데 남의 글이고 남의 서비스에 있다.

방법	되는 것	걸리는 것
직접 긁기	본문	약관, `robots.txt`, 저작권, 개인정보
검색 API	제목, 발췌문, 링크	저장과 수집이 목적이 아니다
모두의 말뭉치	**글 73,176건을 받아 분석**	신청과 승인

다음의 세 페이지에서 첫 번째와 두 번째 방법을 알아본다. 답을 먼저 말하면 **세 번째 방법을 택한다.**

블로그의 robots.txt

https://blog.naver.com/robots.txt (2026-11-03, 줄였다)

```
User-agent: Yeti
Disallow: /
# BOT ACCESS FOR THE PURPOSES OF AI TRAINING AND RETRIEVAL-AUGMENTED GENERATION
↪  (RAG)
# IS STRICTLY PROHIBITED.
User-agent: GPTBot
Disallow: /
User-agent: *
Disallow: /PostList.naver
Disallow: /post/
```

User-agent 어떤 프로그램에게 하는 말인가. *은 나머지 모두다

Disallow 들어오지 말라는 경로의 시작 부분

주석. 학습과 검색 증강에 쓰려는 봇의 접근을 금지한다고 적었다

코드로 읽는 robots.txt

노트북 입력 — 자리표시자 your-id는 자기 것으로 바꾼다

```
HEADERS = {'User-Agent': 'LDS2026-practice/0.1 (your-id@snu.ac.kr)'}
def read_robots(site):
    res = requests.get(site + '/robots.txt', headers=HEADERS, timeout=30)
    rp = robotparser.RobotFileParser()
    rp.parse(res.text.splitlines())
    return rp

naver = read_robots('https://blog.naver.com')
post = 'https://blog.naver.com/someone/123'
print(naver.can_fetch('GPTBot', post))
print(naver.can_fetch('LDS2026-practice', post))
```

```
False
True
```

둘째 줄이 True여도 **허락받았다는 뜻이 아니다.** 약관과 저작권은 따로 있다.

검색 API의 응답

공식 문서의 응답 예시 (줄였다)

```
"total": 56692992, "start": 1, "display": 2,
"items": [{
    "title": "<b>커피</b> 원두 고르는 법과 홈카페 입문 팁",
    "link": "https://blog.naver.com/{blogId}/{logNo}",
    "description": "오늘은 <b>커피</b> 원두를 고르는 기준을 정리해 봤습니다. ... ",
    "bloggername": "{블로그 이름}", "postdate": "20260611"}]
```

description 포스트 내용을 요약한 **발췌문**이다. 본문이 아니다

start 1,000까지, display는 100까지다. total이 5,669만이어도 검색어마다 앞의 1,100건쯤에서 멈춘다

sort 정확도순 또는 날짜순. 검색 엔진이 고른 결과이지 무작위 표본이 아니다

출처: NAVER API HUB 블로그 검색 (https://api.ncloud-docs.com/docs/naver-api-hub-search-blog). 키가 없어 직접 호출하지 않았다.

2026년에 바뀐 것

때	일
2026년 7월	개발자센터의 검색 API 신규 신청 중단, NAVER API HUB로 이전
2026년 9월	검색 API 약관 개정 시행

보도에 따르면 개정 약관은 이런 일을 막는다

검색 결과의 **복사, 저장, 캐싱**(짧은 예외 있음), 제3자 제공, AI 입력과 학습, 평가.

그래서

API로 블로그 글을 모아 코퍼스로 저장하던 지난해의 실습은 하지 않는다. 약관은 바뀌므로 **쓰기 직전에 원문을 읽는다**.

약관: `https://developers.naver.com/products/terms/`. 요약은 2026년 8–9월 보도에 따른 것이다 (한국데이터경제신문, TokenPost, 뉴스웨이).

확인 질문

여기까지 정리

1. 2023–24년 블로그의 말을 연구하려면 어느 자료원을 먼저 찾아갈까? 왜 그럴까?
2. 온라인 대화 말뭉치와 온라인 게시 자료 말뭉치는 무엇이 다른가?
3. 온라인 게시 자료 2025의 표본은 어떻게 뽑았고, 그것을 어디서 알 수 있는가?
4. `can_fetch`가 `True`이면 모아도 되는가?
5. AI Hub 데이터셋을 기말프로젝트에 쓰기 전에 어떤 문구를 확인하는가?

1 자료원 둘러보기

2 **직접 모으기**
- 모으기 전에
- 한 페이지 읽기
- 여러 페이지 모으기
- 코퍼스 점검하기
- 세어 보기

3 남이 다시 할 수 있게

4 마무리

위키문헌을 고른 까닭

물음	어디를 보나	위키문헌 단편소설
써도 되는 글인가	페이지의 라이선스 틀	저작권 보호 기간이 끝났다는 틀
로봇이 들어가도 되나	`robots.txt`	`/wiki/`는 된다, `/w/`는 안 된다
약관과 정책은	로봇 정책	자신을 밝히고, 쿼리 없는 주소로
서버에 부담을 주나	정책, 요청 수	1초 간격, 187번, 파일 재사용
사람을 해치지 않나	내용	옛 작가가 발표한 소설이다

이 다섯 가지 물음이 곧 수집의 조건이다

하나라도 답이 안 나오면 모으지 않는다. 기술적으로 된다는 것은 답이 아니다. 법적인 판단은 이 강의의 범위를 넘으니, 공개할 결과물이라면 전문가에게 묻는다.

출처: `https://wikitech.wikimedia.org/wiki/Robot_policy`

연구 물음과 설계

12강의 물음을 이어서

온라인 대화에서 ‘완전’은 8,636번 혼자 쓰였다. 김다미(2021)에 따르면 ‘완전 다른’의 very는 20세기 말에야 보인다. 그 전의 글에서 ‘완전’은 어떻게 쓰였을까?

설계	오늘의 코퍼스
보고 싶은 것(모집단)	20세기 전반의 한국어 산문
실제로 모을 수 있는 목록(표집틀)	위키문헌 **분류 : 단편 소설**
표본	그 목록 전부
단위와 메타데이터	작품 한 편. 제목, 작가, 연도, 라이선스, 버전

표집틀과 모집단은 다르다

누군가 입력하고 분류한 작품만 들어 있다. 그 차이를 먼저 적어 둔다.

위키문헌의 robots.txt

노트북 입력

```
BASE = 'https://ko.wikisource.org'
AGENT = 'LDS2026-practice'
wiki = read_robots(BASE)
print(wiki.can_fetch(AGENT, BASE + '/wiki/운수_좋은_날'))
print(wiki.can_fetch(AGENT, BASE + '/w/index.php?oldid=468919'))
```

실행 결과

```
True
False
```

oldid는 옛 버전의 주소다. 16강의 revision처럼 쓰고 싶지만 로봇은 들어가지 않는다. 버전은 어떻게 남길까? 뒤에서 푼다.

웹 페이지는 HTML이다

브라우저의 [페이지 소스 보기] — 14강의 XML과 닮은 나무다 (줄였다)

```
<title>운수 좋은 날 - 위키문헌, 우리 모두의 도서관</title>
<div class="mw-content-ltr mw-parser-output" lang="ko" dir="ltr">
  <a href="/wiki/%EC%A0%80%EC%9E%90:..." title="저자:현진건">현진건</a>
  <p>이날이야말로 동소문 안에서 인력거꾼 노릇을 하는 ...</p>
  <div class="licenseContainer ...">...</div>
</div>
<div id="mw-normal-catlinks"> ... <a ...>1924년 작품</a> ...
```

<p> 태그. 14강처럼 열고 닫는다. p는 문단, a는 링크, div는 구역이다

href 속성. 링크의 주소다. %EC%A0%80은 9강의 UTF-8 바이트('저')다

class="..." 스타일을 입히려고 붙인 이름. 원하는 구역을 고르는 손잡이다. 앞에 .을 붙인다

id="..." 한 페이지에 하나뿐인 이름. 앞에 #을 붙인다

한 페이지 내려받기

노트북 입력

```
def fetch(path):
    return requests.get(BASE + path, headers=HEADERS, timeout=30)
r = fetch('/wiki/운수_좋은_날')
print(r.status_code, r.headers['content-type'])
print(r.encoding, len(r.content), len(r.text))
```

실행 결과

```
200 text/html; charset=UTF-8
UTF-8 91501 73699
```

status_code 200은 성공, 404는 없음, 429는 너무 잦음

headers 내가 누구인지 밝힌다. timeout은 기다릴 시간이다

r.content 바이트, r.text는 글자다. 9강에서 본 차이다

원하는 곳만 고르기

노트북 입력

```
soup = BeautifulSoup(r.text, 'html.parser')
body = soup.select_one('div.mw-parser-output')
paras = [p.get_text().strip() for p in body.find_all('p')]
print(len(paras), paras[1][:30])
print(paras[-1][:30])
```

실행 결과

```
88 이날이야말로 동소문 안에서 인력거꾼 노릇을 하는 김 첨
Public domainPublic domainfals
```

마지막 문단은 소설이 아니라 **라이선스 상자**의 글자다. 고르는 범위가 넓으면 이런 것이 함께 온다. `select_one`은 CSS 선택자로 하나, `find_all`은 이름으로 모두 찾는다.

본문이 아닌 것 지우기

노트북 입력 — 나무에서 그 가지를 잘라 낸다

```
for box in body.select('.licenseContainer'):
    box.decompose()
paras = [p.get_text().strip() for p in body.find_all('p')]
paras = [p for p in paras if p]
print(len(paras), paras[-1][:30])
```

실행 결과

```
82 “설렁탕을 사다놓았는데 왜 먹지를 못하니, 왜 먹지를
```

왜 눈으로 보는가

이 문구가 소설로 세어질 뻔했다(약 5만 자). **첫 페이지와 끝 페이지를 읽어 본다.**

페이지는 바뀐다



강의를 준비하던 날 받은 첫 문단이다.
그날 아침 누군가 넣은 장난 편집이다.
위키는 누구나 고친다. 여러분의
화면에서는 이미 사라졌을 수 있다.

2026-11-03 캡처. 사람 이름은 ○○○으로 가렸다.

노트북 입력 — 첫 문단의 줄

```
lines = paras[0].split('\n')
print(len(lines), lines[1][:5])
```

```
10 안녕하세요
```

버전을 남기기

노트북 입력 — 페이지의 소스 안에 버전 번호가 있다

```
print(re.search(r'"wgRevisionId":(\d+)', r.text).group(1))
```

실행 결과

```
468919
```

웹의 재현은 두 겹이다

1. **버전 번호와 내려받은 날을 기록한다.** 수가 다르면 여기부터 비교한다.
2. **받은 HTML을 그대로 둔다.** 파싱을 고칠 때도 다시 받지 않는다.

모두의 말뭉치의 `crawl_date`와 같은 일이다. 버전 번호가 없는 사이트라면 날짜와 원본이 전부다.

목록에서 주소 모으기


"단편소설" 분류에 속하는 문서

다음은 이 분류에 속하는 문서 187개 가운데 187개입니다.

B
- B사감과 러브레터
- B사감과 러브레터(현)

X
- X씨

ㄷ
- ㄷ씨 행장기
- 단발
- 대동강은 속삭인다
- 대동강이 아무리나
- 도시와 유령
- 도정
- 동경 이후기

- 서기서
- 서광이
- 서방
- 서전과 편지
- 선 이(호박)
- 선물
- 선물 니그네
- 선상
- 선생
- 세루홈 도적
- 손귀녀
- 소낙비
- 소년의 비애
- 솔밭이
- 솔밭지

분류 페이지는 작품의 목록이다. 링크를 모두 꺼내면 내려받을 주소가 나온다.

ko.wikisource.org/wiki/분류:단편소설 (2026-11-03 캡처)

노트북 입력

```
page = BeautifulSoup(fetch('/wiki/분류:단편소설').text, 'html.parser')
links = [(a['title'], a['href']) for a in page.select('#mw-pages li a')]
print(len(links), links[1][0])
```

```
187 B사감과 러브레터
```

천천히 모으기

노트북 입력 — 돌려 놓고 설명을 듣는다(3분 넘게 걸린다)

```
for title, href in tqdm(links):
    path = 'data/wikisource/raw/' + title.replace('/', '_') + '.html'
    if os.path.exists(path):
        continue
    res = fetch(href)
    res.raise_for_status()
    with open(path, 'w', encoding='utf-8') as f:
        f.write(res.text)
    time.sleep(1)
```

`time.sleep(1)` 한 번 받고 1초 쉰다. 함께 돌리니 줄이지 않는다

`os.path.exists` 받은 페이지는 건너뛴다. 끊겨도 이어서 할 수 있다

`raise_for_status` 200이 아니면 멈춘다. 오류 페이지를 저장하지 않는다

429를 받으면 응답의 `Retry-After`만큼 기다린다.

한 페이지를 한 행으로

```
def parse(path):
    with open(path, encoding='utf-8') as f:
        html = f.read()
    soup = BeautifulSoup(html, 'html.parser')
    body = soup.select_one('div.mw-parser-output')
    for box in body.select('.licenseContainer'):
        box.decompose()
    paras = [p.get_text().strip() for p in body.find_all('p')]
    cats = [a.get_text() for a in soup.select('#mw-normal-catlinks li a')]
    years = [c[:4] for c in cats if re.fullmatch(r'\d{4}년 작품', c)]
    return {'title': path.split('/')[-1].removesuffix('.html'),
            'author': body.select_one('a[title^="저자:"]').get_text(),
            'year': int(years[0]) if years else None,
            'license': ' '.join(c for c in cats if c.startswith('PD-')),
            'rev': int(re.search(r'"wgRevisionId":(\d+)', html).group(1)),
            'text': '\n'.join(p for p in paras if p)}
```

앞에서 한 일을 함수로 묶었다. `a[title^="저자:"]`는 `title`이 '저자:'로 시작하는 `a`다.

코퍼스의 첫 표

노트북 입력 — 내려받은 파일만 읽는다

```
paths = sorted(glob('data/wikisource/raw/*.html'))
docs = pd.DataFrame([parse(p) for p in paths])
docs['year'] = docs['year'].astype('Int64')
print(docs.shape)
print(docs[['title', 'author', 'year', 'license', 'rev']].head(3))
```

실행 결과

```
(187, 6)
       title author  year    license     rev
0     B녀의 소묘    이무영  1934  PD-old-50  384422
1  B사감과 러브레터    현진건  1925  PD-old-50  464471
2         X씨    김동인  1925  PD-old-70  385642
```

`astype('Int64')`는 빈칸(<NA>)을 품을 수 있는 정수다.

무엇이 들어 있나

노트북 입력

```
print(docs['author'].value_counts().head(3).to_dict())
decades = pd.cut(docs['year'], [1900, 1919, 1929, 1939, 1959],
                 labels=['~1919', '1920s', '1930s', '1940~'])
print(decades.value_counts(sort=False).to_dict())
print(docs['year'].isna().sum())
```

```
{'김동인': 47, '김유정': 17, '강경애': 17}
{'~1919': 9, '1920s': 24, '1930s': 71, '1940~': 9}
74
```

작가 넷 중 하나가 김동인이다. 당시 문단이 아니라 **입력한 사람의 선택**이다

연도 없음 74페이지. 시기별로 나누면 그만큼 빠진다

분류 김유정론(작가론)과 번역 동화도 있다. 분류도 사람이 붙인다(16강)

다듬은 내용을 기록하며

노트북 입력

```
docs = docs[~docs['title'].str.startswith('번역:')].copy()
JUNK = r'안녕하세요 \S+입니다\n?'
docs['text'] = docs['text'].str.replace(JUNK, '', regex=True)
docs['chars'] = docs['text'].str.len()
print(len(docs), docs['chars'].sum())
```

실행 결과

```
186 2131049
```

지우지 않고 둔 것도 적는다

편집자 주석(‘출전: 동아일보…’), 한자 병기, 옛 맞춤법, 이상의 붙여쓰기(**지주회시**).

‘아주’와 ‘완전’을 센다

노트북 입력

```
text = docs['text']
pats = ['아주', r'\b아주\b', '아주머니',
        '완전', r'\b완전\b',
        '완전히', r'\b완전히\b']
for pat in pats:
    print(pat, text.str.count(pat).sum())
```

실행 결과

```
아주 362
\b아주\b 212
아주머니 69
완전 71
\b완전\b 0
완전히 57
\b완전히\b 51
```

아주머니 \b가 ‘아주머니’ 69번을 막아 준다

완전 0 186편에서 ‘완전’은 한 번도 **혼자** 쓰이지 않았다

완전히잊어 이상의 붙여쓰기 때문에 ‘완전히’는 문자열로 센다

온라인 대화와 나란히

노트북 입력 — 12강의 표를 다시 읽어 100만 자당으로

```
PATS = {'완전': r'\b완전\b', '완전히': '완전히', '아주': r'\b아주\b'}
def per_million(s):
    return {w: s.str.count(p).sum() / s.str.len().sum() * 1e6
            for w, p in PATS.items()}
u = pd.read_csv('data/nikl/utterances.csv')
table = pd.DataFrame({'단편소설': per_million(text),
                      '온라인 대화': per_million(u['form'])})
print(table.round(1))
```

```
      단편소설  온라인 대화
완전     0.0   217.1
완전히  26.7     4.5
아주    99.5    85.4
```

‘아주’는 비슷하다. ‘완전’과 ‘완전히’는 **자리가 뒤집힌다**. 크기가 다른 코퍼스는 비율로 비교한다(8강).

이 수로 무엇을 말할까

말할 수 있는 것

위키문헌의 이 단편소설들에서 '완전'은 '완전히', '완전한'처럼 쓰이고, 혼자 쓰인 예는 없다. 온라인 대화의 비율대로라면 213만 자에 460번쯤 나올 말이다. 김다미(2021)가 그린 시기 구분과 잘 맞는 수다.

말할 수 없는 것

'1930년대 사람들은 완전을 부사로 쓰지 않았다'는 아니다. 당시의 말은 여기 없다. 문체, 매체, 시기가 한꺼번에 다르므로 두 칸의 차이를 시기 하나로 돌릴 수 없다.

그다음에 할 일

같은 시기 신문이나 잡지, 같은 매체의 다른 시기를 더한다. 세 가지 변수를 하나씩 떼어 보는 것이 코퍼스 설계다.

작가마다 다르다

노트북 입력

```
docs['아주'] = text.str.count(r'\b아주\b')
by = docs.groupby('author')[['chars', '아주']].sum()
by['n'] = docs['author'].value_counts()
by['per_100k'] = (by['아주'] / by['chars'] * 1e5).round(1)
print(by[by['n'] >= 15].sort_values('per_100k', ascending=False))
```

```
          chars  아주   n  per_100k
author
김유정     174862  40  17      22.9
강경애     173370  24  17      13.8
이상       160473  22  15      13.7
김동인     515984  32  47       6.2
윤백남     189990   5  15       2.6
```

작품이 많은 작가 다섯 명 사이에서도 10만 자당 수가 아홉 배 가까이 차이 난다. 13강에서 사람마다 다시 세었던 것과 같다.

1 자료원 둘러보기

2 직접 모으기

3 남이 다시 할 수 있게
- 코퍼스 카드
- 직접 해 보기
- 저장소에 남기기

4 마무리

저장하기

노트북 입력

```
docs.to_csv('corpus/wikisource/docs.csv', index=False)
back = pd.read_csv('corpus/wikisource/docs.csv')
print(back.shape, back['text'].str.len().sum())
```

```
(186, 8) 2131049
```

어디에	무엇	git에
data/wikisource/raw/	받은 HTML 그대로	올리지 않는다
corpus/wikisource/docs.csv	다듬은 표	올린다
corpus/wikisource/README.md	코퍼스 카드	올린다

저작권이 끝난 글이라 표를 올린다. 모두의 말뭉치, AI Hub, 블로그의 글은 올리지 않는다.

코퍼스 카드 corpus/wikisource/README.md

```
# 위키문헌 단편소설 코퍼스
- 출처: 한국어 위키문헌 `분류:단편소설` 187페이지, 2026-11-11 내려받음
- 규모: 186편, 약 213만 자. 작품 한 편이 한 행(`docs.csv`)
- 열: title, author, year, license, rev(버전 번호), text, chars
- 만든 과정: `17-corpus.ipynb`. `robots.txt` 확인, 1초 간격, `/wiki/제목`만 요청
- 뺀 것: 번역 1편, 라이선스 상자, 장난 편집 `안녕하세요 ...입니다` 줄
- 그대로 둔 것: 편집자 주석, 옛 맞춤법, 이상의 붙여쓰기, 작가론 등 소설 아닌 글
- 치우침: 입력된 작품만 있다. 김동인 47편(25%), 연도 없는 페이지 74편
- 쓰지 말 것: 이 표로 당시 한국어 전체나 시기 변화를 단정하지 않는다
- 라이선스: 작품은 페이지의 틀(PD-old-50/70), 위키문헌의 내용은 CC BY-SA
- 인용: 위키문헌 기여자들. 페이지마다 `rev`로 버전을 밝힌다
```

16강의 일곱 가지 물음에 답한 것이다

무엇을, 누가 어떻게, 어떻게 나누어(이번엔 나누지 않았다), 불러오기, 버전, 써도 되나, 읽어 보았나. 카드의 빈칸은 내가 채운다. [Needs More Information]은 없다.

여기서 멈춘다

만나는 것	할 일
로그인해야 보이는 글	가져오지 않는다. 약관을 읽고 관리자에게 묻는다
CAPTCHA, IP 차단, 429	막은 것이다. 우회하지 않고 멈춘다
`User-Agent`를 브라우저로 위장	하지 않는다. 자신을 밝힌다
`requests`로는 본문이 비어 있음	자바스크립트가 그린 페이지이다. API나 다른 자료원을 먼저 찾는다
실명, 연락처, 얼굴, 건강 정보	모으지 않는다. 필요하면 IRB에 묻는다

공개되어 있다고 내 것은 아니다

볼 수 있는 글과 모아서 다시 내놓아도 되는 글은 다르다. 글쓴이는 내 연구를 예상하지 않았다. 모두의 말뭉치가 글쓴이를 모집하고 이름을 가린 것은 이 때문이다. 내 코퍼스도 같은 물음에 답해야 한다.

실습

정도 부사를 비교해 보자

오늘 만든 `text`로 **아주**, **매우**, **몹시**, **퍽**, **대단히**, **너무**를 비교한다.

1. 단어마다 `\b`로 전체 횟수를 센다.
2. 단어마다 몇 편에 나오는지 센다.
3. 횟수와 편 수의 순서가 같은가? 다르면 무엇을 뜻하는가?
4. 가장 잦은 단어 하나의 용례를 다섯 개 읽고, 오늘날의 쓰임과 같은지 말해 보자.

오늘 쓴 코드를 고쳐 쓰면 된다. 서버에 다시 요청할 일은 없다. 5분쯤 해 본 다음 답을 함께 본다.

실습 풀이

노트북 입력

```
words = ['아주', '매우', '몹시', '퍽', '대단히', '너무']
print({w: int(text.str.count(rf'\b{w}\b').sum()) for w in words})
print({w: int(text.str.contains(rf'\b{w}\b').sum()) for w in words})
```

실행 결과

```
{'아주': 212, '매우': 83, '몹시': 196, '퍽': 111, '대단히': 33, '너무': 221}
{'아주': 90, '매우': 49, '몹시': 86, '퍽': 51, '대단히': 28, '너무': 96}
```

count 페이지마다 횟수를 더한다

contains 페이지마다 있으면 True. 합이 곧 편 수다

8강의 문서 빈도다. 두 순서가 같다. 편당 횟수는 '너무' 2.3, '대단히' 1.2다. 수의 뜻은 용례를 읽어야 안다. 옛 소설의 '너무'는 무엇과 함께 쓰일까?

기말프로젝트의 자료원을 고를 때

보고 싶은 말	먼저 찾아갈 곳	확인할 것
블로그, 게시판의 글	온라인 게시 자료 말뭉치	신청 시간, 매체의 치우침
메신저 대화	온라인 대화 말뭉치, AI Hub 한국어 SNS	용도 조항, 신청 자격
일상 대화	일상 대화 말뭉치(13강)	말한 사람의 정보
공문, 보도자료	정책브리핑, 공공데이터포털	공공누리 유형
지난날의 글	위키문헌	입력된 작품만 있다는 치우침
분류용 데이터	HuggingFace(16강)	점검표 일곱 가지 물음

중간발표에서

[숙제12]의 데이터 페이지에 자료원, 얻는 절차, 조건을 적자. 아직 승인을 기다린다면 그 사실도 적는다.

무엇을 남길까

터미널 입력 — 노트북이 목록에 없으면 Ctrl+S로 저장부터 한다

```
git status
```

실행 결과

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        17-corpus.ipynb
        corpus/

nothing added to commit but untracked files present (use "git add" to track)
```

받은 HTML은 data/에 있어 보이지 않는다. 카드를 안 썼으면 지금 만든다.

커밋하기

터미널 입력

```
git add 17-corpus.ipynb corpus/wikisource
git commit -m "17강 실습: 위키문헌 단편소설 코퍼스와 카드"
```

실행 결과

```
[main a1b2c3d] 17강 실습: 위키문헌 단편소설 코퍼스와 카드
 3 files changed, 43332 insertions(+)
 create mode 100644 17-corpus.ipynb
 create mode 100644 corpus/wikisource/README.md
 create mode 100644 corpus/wikisource/docs.csv
```

줄 수가 많은 것은 docs.csv에 소설 본문이 들어 있기 때문이다. 키나 비밀번호를 코드에 적었다면 커밋하기 전에 지운다. 커밋하면 역사에 남는다(2강).

푸시하기

터미널 입력

```
git push
```

실행 결과

```
Enumerating objects: 8, done.
Writing objects: 100% (7/7), 1.83 MiB | 8.75 MiB/s, done.
To https://github.com/your-username/LDS2026.git
   e4f5a6b..a1b2c3d  main -> main
```

재현에 필요한 것이 모두 남았다

주소, 요청 간격, 파싱 규칙, 다듬은 내용, 페이지마다의 버전, 카드가 남았다. 웹은 바뀌어 코드만으로 같은 표가 나온다는 보장이 없으니 `docs.csv`도 남겼다.

1 자료원 둘러보기

2 직접 모으기

3 남이 다시 할 수 있게

4 **마무리**
요약
다음 시간
과제

오늘 쓴 것

하는 일	오늘의 코드
규칙 읽기	`robotparser`, `rp.can_fetch(agent, url)`
한 페이지 내려받기	`requests.get(url, headers=HEADERS, timeout=30)`
응답 확인	`r.status_code`, `r.raise_for_status()`
나무 만들기	`BeautifulSoup(r.text, 'html.parser')`
고르기	`soup.select_one('div.x')`, `soup.select('#y li a')`
글자와 속성	`p.get_text()`, `a['href']`
가지 지우기	`box.decompose()`
천천히, 이어서	`time.sleep(1)`, `os.path.exists(path)`

표로 바꾼 뒤로는 12강의 `\b`, 7강의 `groupby`, 8강의 비율 그대로다.

오늘 배운 것

자료원

- 공개 → 신청 → API → 직접 모으기
- 온라인 게시 자료 2025에 블로그 글
- AI Hub는 용도와 자격을 먼저 읽는다

읽기

- 표집틀은 모집단이 아니다
- 본문 아닌 것이 섞인다. 눈으로 본다
- 혼자 쓴 '완전': 0 대 온라인 대화 217

모으기

- 라이선스, 규칙, 약관, 부담, 사람
- 자신을 밝히고, 천천히, 원본을 둔다

남기기

- 페이지마다 버전과 내려받은 날
- 다듬은 것과 안 다듬은 것
- 카드에는 치우침과 쓰지 말 것까지

확인 질문

오늘 것을 정리해 봅시다

1. 크롤링 전에 찾아볼 다른 방법은 무엇이고, 왜 먼저 찾는가?
2. 코드에 `time.sleep`과 `os.path.exists`가 있는 까닭은 각각 무엇인가?
3. `oldid`로 받지 않은 까닭은? 그러면 버전을 어떻게 남겼는가?
4. 표집틀과 모집단이 다르다는 것을 오늘의 코퍼스로 설명해 보자.
5. '완전' 0을 시기 변화의 증거로 쓰면 무엇이 문제일까?
6. 내 기말프로젝트의 자료원은 사다리의 몇 번째 칸인가? 카드의 어느 줄이 비는가?

다음 시간에 할 것

11월 16일(월), 18일(수) 기말프로젝트 중간발표

연구 물음, 자료원, 지금까지 얻은 수를 발표한다. 친구의 자료원에 대해 오늘의 물음을 던져 보자. 표집틀은 무엇인가? 써도 되는가?

제18강 기계학습 (1) 로지스틱 회귀분석을 사용한 분류 (11월 23일 월요일)

지금까지 사람이 세었다. 이제 모델에게 규칙을 배우게 한다. 16강의 '부정이면 모순'을 기억해 두자.

과제 안내

[숙제12] 기말프로젝트 중간발표 자료 [보고서]

마감 **2026-11-13 12:00 (금요일 정오)**. 발표는 11월 16일(월)과 18일(수) 두 번에 나누어 한다. 데이터를 소개하는 페이지에 자료원, 얻는 절차, 조건과 16강의 점검표를 넣자.

[숙제00심화] 논문의 통계량·시각화 재검토 (선택)

읽은 논문이 어떤 표집틀에서 무엇을 모았는지 살펴보자. eTL 심화과제 게시판에 공개글로, 12월 18일 09:00까지 수시로 올린다.